Algoritmos y Estructuras de Datos

Problem Sets UT1.

Los ejercicios obligatorios son:

Ejercicio 7 – Ejercicio 10 – Ejercicio 12

Ejercicio 1

Analiza el orden del tiempo de ejecución de los siguientes algoritmos

a)

```
public static int enRango (int[] a, int bajo, int alto) {
   int contador = 0;
   for (int i = 0; i < a.length; i++) {
      if (a[i] >= bajo && a[i] < alto)
         contador++;
   }
   return contador;
}
```

b)

```
unaFunción(N de tipo entero)
   i <- 1
   j <- N
   mientras i < N hacer
      j <- N - 1
      i <- i * 2
   fin mientras
   devolver (j)
fin
```

c)

```
int[] cuentas = new int[100];
for (int i = 0; i < 100; i++) {
   cuentas[i] = enRango(notas , i, i+1);
}
```

Ejercicio 2

Analiza el orden del tiempo de ejecución del siguiente algoritmo:

```
unValor (A, N de tipos enteros)
      i <- 0
      Si N < 3 entonces
         devolver (A)
      fin si
      mientras i < 3 hacer
         si arreglo[i] = A entonces
            devolver ((arreglo[0] + arreglo[N-1]) div 2)
         fin si
         i <- i + 1
      fin mientras
      devolver (A div N)
Fin
```

Ejercicio 3

Analiza el orden del tiempo de ejecución del siguiente algoritmo:

```
otraFunción(claveAbuscar)
      inicio <- 0
      fin <- N - 1
      mientras inicio <= fin hacer
         medio <- (inicio + fin) div 2
         si (arreglo[medio] < claveAbuscar) entonces
            inicio <- medio + 1
         sino
            si (arreglo[medio] > claveAbuscar) entonces
               fin <- medio - 1
            Sino
                devolver medio
            fin si
         fin si
      fin mientras
      devolver -1
fin
```

Ejercicio 4

Analiza el orden de tiempo de ejecución del siguiente algoritmo:

```
function particion(i, j: integer; pivote: integer): integer;
/* Divide V[i], .., V[j] para que las claves menores que pivote estén
a la izquierda y las mayores o iguales a la derecha.
Devuelve el lugar donde se inicia el grupo de la derecha.
*/

COMIENZO
   L <- i;
   R <- j;
   Repetir
      intercambia(V[L], V[R]);
      mientras V[L] < pivote hacer L := L + 1; fin mientras
      mientras V[R] >= pivote hacer R := R - 1; fin mientras
   Hasta que L > R
   Devolver L;
FIN
```

Ejercicio 5

Analiza el orden de tiempo de ejecución del siguiente algoritmo:

```
miFunción
      Desde i = 1 hasta N-1 hacer
         Desde j = N hasta i+1 hacer
            Si arreglo[j] < arreglo[j-1] entonces
               Intercambia(arreglo[j], arreglo[j-1])
            Fin si
         Fin desde
      Fin desde
Fin
```

Ejercicio 6

Analiza el orden de tiempo de ejecución del siguiente algoritmo:

```
Desde i = 1 hasta N-1 hacer
   Desde j = N hasta i+1 hacer
      Si arreglo[j] < arreglo[j-1] entonces
          Intercambia(arreglo[j], arreglo[j-1])
      FinSi
   FinDesde
FinDesde
```

1. Analiza el orden de complejidad temporal:
    a) mejor caso
    b) peor caso

2. Justifica el resultado.

Ejercicio 7

Implementa en **Java** el algoritmo del ejercicio anterior, usando un contador para contar cuántas veces se va a invocar la sentencia **Si**. El programa debe:

Leer de un archivo numeros.txt cuya primera línea será la cantidad de números a leer (sea N) y las siguientes N líneas contendrán los elementos de entrada del algoritmo, uno por cada línea.

El programa debe mostrar:

- El valor de N
- El **contenido del contador**
- La cantidad de **intercambios realizados**
- El primer y el último elemento del arreglo resultante

Ejercicio 8

A) ¿Qué hace el siguiente código¿

```
public void myFunction(int i) {
       Nodo nodo = new Nodo(i, this.primero);
       cabeza = nodo;
}
```

a. Agregar un nodo a continuación del nodo “cabeza”.
b. Quitar el nodo de la lista.
c. Agregar un nodo al principio de la lista.
d. Agregar un nodo al final de la lista.

B) En una lista circular, el siguiente fragmento de código Java:

```
actual = this.primero;
while (actual.siguiente != null)
   actual = actual.siguiente;

actual.siguiente = otroNodo;
otroNodo.siguiente = null;
```

Posibles interpretaciones:

a. Puede arrojar un resultado correcto si la lista fuera doblemente enlazada.

b. Debe necesariamente usar un nodo especial de cabecera para que el resultado sea correcto.
c. Puede dar un error en tiempo de ejecución si actual termina siendo null al terminar el bucle.
d. Aunque actual no sea null al inicio, ejecuta indefinidamente.

C) En una lista enlazada simple, el siguiente fragmento de código Java:

```
temporal.siguiente = actual.siguiente;
actual.siguiente = temporal;
```

Posibles interpretaciones:

a. Elimina de la lista al nodo actual.
b. Elimina de la lista al nodo temporal.
c. Inserta el nodo actual a continuación del nodo temporal.
d. Inserta el nodo temporal a continuación del nodo actual.

D) En una lista enlazada simple, el siguiente fragmento de código Java:

```
temporal = actual.siguiente;
actual.siguiente = temporal.siguiente;
```

Posibles interpretaciones:

a. Elimina de la lista al nodo temporal.
b. Elimina de la lista al nodo actual.
c. Inserta el nodo actual a continuación del nodo temporal.
d. Inserta el nodo temporal a continuación del nodo actual.

Ejercicio 9

La Biblioteca Central UCU necesita gestionar su catálogo de libros y nos encarga construir un sistema eficiente. Por cada libro se almacenará:

- Título del libro (máx. 30 caracteres).
- Código de identificación (ISBN o código interno, máx. 20 caracteres).
- Precio de reposición.

La primera versión del sistema deberá implementar:

- Incorporar un nuevo libro al catálogo.
- Agregar ejemplares (stock) a un libro existente.
- Reducir ejemplares (stock) de un libro existente (préstamo o baja).
- Listar todos los libros ordenados por título, mostrando además su stock.

Las listas pueden implementarse con arrays o listas encadenadas. Analizar:

1. Costo de memoria en cada caso.
2. Consideraciones sobre la cantidad de libros soportada.
3. Consideraciones de eficiencia para cada operación según la implementación.

Ejercicio 10

La/el Arquitect@ decidió implementar con listas con encadenamiento simple y solicita desarrollar un algoritmo de alta de un nuevo libro. Requisitos:

- Descripción de alto nivel.
- Precondiciones y postcondiciones.
- Seudocódigo detallado.

Ejercicios 11

Los nodos de una lista simplemente encadenada tienen dos atributos:

- DATOS: de tipo dato.
- SIGUIENTE: de tipo nodo de lista, que hace referencia al siguiente nodo.

Sean nodo1, nodo2 y nodo3 tres nodos consecutivos (nodo2 sigue a nodo1, nodo3 sigue a nodo2), y el siguiente fragmento de código:

```
// en algún lado se crea el nodo “otroNodo”
otroNodo.siguiente = nodo1
nodo2.siguiente = nodo3
```

¿Qué opción es la correcta?

1. Inserta otroNodo antes de nodo1.
2. Inserta otroNodo entre nodo1 y nodo2.
3. Elimina nodo2.
4. No tiene efecto.

Ejercicio 12

Dado el siguiente fragmento de código:

```
otroNodo <- nodo1.siguiente
nodo1.siguiente <- nodo3
```

Opciones:

1. Inserta otroNodo antes de nodo1.
2. Inserta otroNodo entre nodo1 y nodo2.
3. Elimina nodo2.

4. No tiene efecto.

Ejercicio 13

Dado el siguiente fragmento de código y el nodo “otroNodo”

```
otroNodo.siguiente <- nodo1.siguiente
nodo1.siguiente <- otroNodo
```

¿Qué opción es correcta?

- Inserta otroNodo antes de nodo1.
- Inserta otroNodo entre nodo1 y nodo2.
- Elimina nodo2.
- Error en ejecución si nodo1 es el primero o nodo3 es el último.

Ejercicio 14

Dado el siguiente fragmento de código y los nodo “otroNodo” y “nodoActual”:

```
nodoActual <- primero
mientras nodoActual <> nulo hacer
      nodoActual = nodoActual.siguiente
fin mientras
nodoActual.siguiente = otroNodo
```

¿Qué opción es correcta?

1. Inserta otroNodo como último nodo.
2. Inserta otroNodo como primer nodo.
3. Error si la lista está vacía.
4. Siempre error.

Ejercicio 15

Dado el siguiente fragmento de código y los nodo “otroNodo” y “nodoActual”:

```
nodoActual <- primero
mientras nodoActual.siguiente <> nulo hacer
      nodoActual <- nodoActual.siguiente
fin mientras
nodoActual.siguiente <- otroNodo
```

Opciones:

1. Inserta otroNodo como último nodo.

2. Inserta otroNodo como primer nodo.
3. Error si la lista está vacía.
4. Siempre error.

Ejercicio 16

Se desea llevar un registro de asistencia de un curso universitario con cantidad indeterminada de alumnos, representados en una lista. Cada elemento:

- ID del alumno.
- Contador de asistencias.

Además:

- Registrar el total de clases impartidas.
- Calcular el % de asistencia por alumno.

Analizar implementación con:

- Array.
- Lista encadenada.

Preguntas:

1. Costo de memoria.
2. Consideraciones sobre cantidad de alumnos soportada.

Ejercicio 17

Escenario

La Biblioteca Central UCU necesita gestionar su catálogo y existencias (ejemplares disponibles para préstamo) y nos encarga construir un sistema eficiente. Por cada libro se almacenará:

Título del libro.

- Código de identificación (ISBN o código interno).
- Precio de reposición.
- Cantidad de ejemplares disponibles.

El sistema deberá permitir:

1.Incorporar un nuevo libro al catálogo.

2. Agregar ejemplares a un libro existente.

3. Registrar préstamo o devolución de un libro.

4. Retirar del catálogo libros que ya no circulan.

5. Consultar existencias de un libro por su código.

6. Listar todos los libros, ordenados por título, con su stock

Paso 1 – Clase en común

1. Completar los métodos de inserción y búsqueda en el TDA Lista.
2. Diseñar las estructuras de datos utilizando el TDA Lista.
3. Declarar las clases Biblioteca y Libro según las funcionalidades dadas

Paso 2 – Implementar en sub-equipos

Sub-equipo A – Adquisiciones: Registrar compra o donación de ejemplares, o alta de un libro nuevo, e indicar el valor total agregado al stock.

Formato del archivo adquisiciones.txt:

```
CODIGO_LIBRO, TITULO_LIBRO, PRECIO_REPOSICION, CANTIDA
```

Sub-equipo B – Préstamos: Registrar préstamos (reducen stock) y devoluciones (aumentan stock) e indicar la variación total de ejemplares prestados.

Formato del archivo prestamos.txt:

```
CODIGO_LIBRO, TIPO, CANTIDAD
```

donde TIPO es PRESTAMO o DEVOLUCION.

Nota: si la cantidad solicitada en un préstamo excede el stock, se presta hasta donde alcance.

Paso 3 - Integración y verificación

Integrar el código de todos los sub-equipos en un único programa que procese:

```
adquisiciones.txt
prestamos.txt
```

*Nota: Los archivos deben estar bajo **/src/main/resources***

Ejercicio 18

En nuestro TDA Lista, además de insertar y buscar elementos, eventualmente será necesario retirar un elemento de la misma.

Se podrían definir los siguientes comportamientos:

1. Quitar un elemento de la lista (si existe) y dejarlo disponible para su posterior utilización.

2. Quitar y destruir un elemento (podría llamarse “Eliminar”).

3. Otros comportamientos posibles (a definir).

Consigna

Discutir en el equipo la conveniencia de estos comportamientos y determinar:

- Qué forma deberían tener (qué parámetros reciben como entrada).
- Qué valor o resultado deberían devolver.

Sugerencia

Analizar:

- Qué sucede con el campo ***Siguiente*** de un elemento que se ha retirado de la lista.
- Qué precauciones deberían tomarse para evitar errores o referencias inválidas.